LeetCode Practice

3 Hard: Bit Manipulation

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

3
HARD

Patterns (in order)

1. Bit Manipulation (3)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

3 questions | tap a title to jump to it

Hard questions (3) Go >

Bit Manipulation (3) Go >

H 1. Optimal Account Balancing LC 465

H 2. Shortest Path Visiting All Nodes LC 847

H 3. Unique Paths III LC 980

Hard questions (3)

HARD | TOPIC 1 OF 1

Bit Manipulation

3 Hard questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Optimal Account Balancing LC 465
2. Shortest Path Visiting All Nodes LC 847
3. Unique Paths III LC 980

HARD | Bit Manipulation | LeetCode #465 | 1 of 3 | 1/3

Optimal Account Balancing

QUESTION

You are given an array of transactions transactions where transactions[i] = [from$_i$, to$_i$, amount$_i$] indicates that the person with ID = from$_i$ gave amount$_i$ $ to the person with ID = to$_i$.

Return the minimum number of transactions required to settle the debt.

Example 1	
Input	`transactions = [[0,1,10],[2,0,5]]`
Output	**2**
Explanation	*Person #0 gave person #1 $10. Person #2 gave person #0 $5. Two transactions are needed. One way to settle the debt is person #1 pays person #0 and #2 $5 each.*

Example 2	
Input	`transactions = [[0,1,10],[1,0,1],[1,2,5],[2,0,5]]`
Output	**1**
Explanation	*Person #0 gave person #1 $10. Person #1 gave person #0 $1. Person #1 gave person #2 $5. Person #2 gave person #0 $5. Therefore, person #1 only need to give person #0 $4, and all debt is settled.*

CONSTRAINTS

- 1 <= transactions.length <= 8
- transactions[i].length == 3
- 0 <= from$_i$, to$_i$ < 12
- from$_i$!= to$_i$
- 1 <= amount$_i$ <= 100

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Settle debts between people with minimum transactions. Right?"

"A few quick questions: see constraints."

Can the matrix be empty or 1×1? Any constraints on its dimensions?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
net[person] = paid minus owed from all records
def settle(i):
    skip people with zero balance
    for j in js:
        # with the opposite sign
        transfer, recurse, undo
    return the fewest transfers
```

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

Tries every settlement order. Only people with a non-zero balance matter; backtracking with pruning (skip equal balances, stop early when a person settles exactly) or bitmask DP over subsets that sum to zero is O(n * 2^n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Brute-force permutations or DP — O(n * 2^n). Walk-through:"*

Net debt array: for each person, compute net[i] = total_received - total_paid.

Only non-zero net balances matter. Problem reduces to: find min transactions to settle debts.

Backtracking: try all subsets of people. If a subset has zero total debt, one transaction

settles that subset (|subset|-1 transactions for |subset| people).

Alternatively: match creditors with debtors greedily (approximation) or exactly (DP).

DP: dp[mask] = min transactions to settle the debt subset defined by mask.

For each submask: if sum(net[i] for i in submask) == 0: dp[mask] = min(dp[mask], dp[mask^submask]+1).

WHY zero-sum subsets: internally settle before involving others.

Time: O(n * 2^n). Space: O(2^n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _465_OptimalAccountBalancing:
    def minTransfers(self, transactions):
        from collections import defaultdict
        balance = defaultdict(int)
        for a, b, amount in transactions:
            balance[a] -= amount; balance[b] += amount
        debts = [v for v in balance.values() if v != 0]
        n = len(debts)
        memo = {}
        def dfs(mask):
            if mask in memo: return memo[mask]
            total = sum(debts[i] for i in range(n) if mask
```

```
>>i&1)
            if total == 0:
                memo[mask] = bin(mask).count('1') - 1
                return memo[mask]
            memo[mask] = float('inf')
            sub = (mask-1)&mask
            while sub:
                if sum(debts[i] for i in range(n) if sub>>
i&1)==0:
                    memo[mask]=min(memo[mask],dfs(sub)+dfs
(mask^sub))
                sub=(sub-1)&mask
            return memo[mask]
        return dfs((1<<n)-1)
```

PHASE 5 TEST & DEBUG

"Let me trace minTransfers([[0,1,10],[2,0,5]])."

Net balances: person0: -10+5=-5, person1: +10, person2: -5

(person0 owes 5, person1 is owed 10, person2 owes 5)

Simplify: debts=[-5,-5,10]

DFS: match -5(idx0) with +10(idx2): give 5 -> debts=[-5,0,5] -> 1 transaction

Match remaining -5(idx0) with +5(idx2): give 5 -> debts=[0,0,0] -> 2 transactions

Return 2 ok

Key insight: NP (Non-deterministic Polynomial)-hard to minimize transactions optimally; small n (<=12) makes exhaustive DFS with pruning feasible.

Prune: skip zero-balance debts; try matching each positive debt.

dp[mask]=min transactions to settle accounts in subset mask. dp[0]=0. dp[i] computed via bitmask enumeration. dp[j]=dp[j^(1<<k)]+1 when balance[j] and balance[k] cancel.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n*2^n)	**O(n)**
Subset DP on balanced groups."	DP table, reducible to O(1) rolling window for 1-D recurrences

Follow-up: what if you want the actual sequence of transactions, not just the minimum count? Store the transaction pairs during backtracking.

HARD | Bit Manipulation | LeetCode #847 | 2 of 3 | 2/3

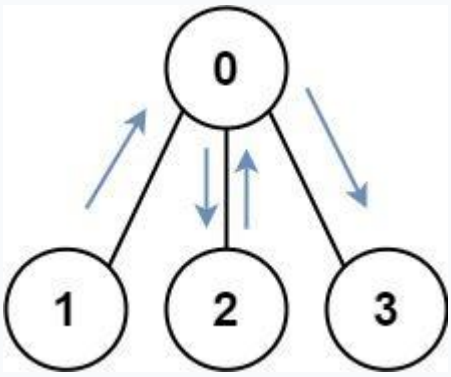
Shortest Path Visiting All Nodes

QUESTION

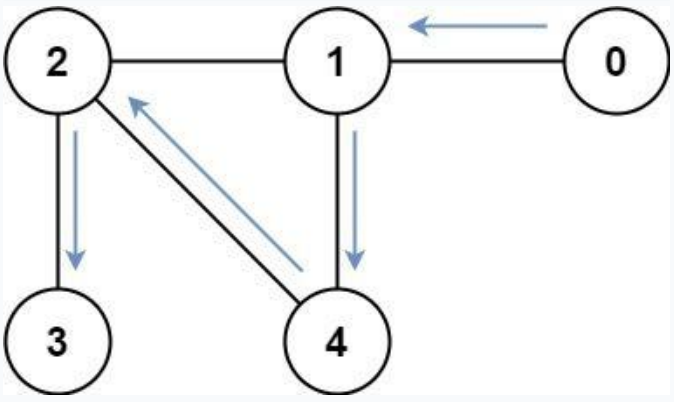
You have an undirected, connected graph of n nodes labeled from 0 to n - 1. You are given an array graph where graph[i] is a list of all the nodes connected with node i by an edge.

Return the length of the shortest path that visits every node. You may start and stop at any node, you may revisit nodes multiple times, and you may reuse edges.

Example 1	
Input	`graph = [[1,2,3],[0],[0],[0]]`
Output	`4`
Explanation	*One possible path is [1,0,2,0,3]*



Example 2	
Input	`graph = [[1],[0,2,4],[1,3,4],[2],[1,2]]`
Output	`4`
Explanation	*One possible path is [0,1,4,2,3]*



CONSTRAINTS

- n == graph.length
- 1 <= n <= 12
- 0 <= graph[i].length < n
- graph[i] does not contain i.
- If graph[a] contains b, then graph[b] contains a.
- The input graph is always connected.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the shortest path that visits all nodes in an undirected graph. Can revisit nodes. Right?"

"A few quick questions:

Return the minimum number of edges traversed? Yes."

"Let me walk: graph=[[1,2,3],[0],[0],[0]] -> 4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(path):
    from every start node, try every walk and
        stop when all nodes have been visited
    best = min(best, candidate)
```

TIME	SPACE
O(n!)	O(n)

WHY IT IS SLOW

Tries every walk and revisits states. BFS over (node, bitmask of visited nodes) states, starting from every node at once: O(2^n * n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"BFS on (node, visited_mask) state — already O(2^n * n). Walk-through:"*

State = (current_node, bitmask of visited nodes).

Initial states: (i, 1<<i) for each node i (can start at any node, visited = just i).

BFS queue processes states in order of steps (edges traversed).

For each state (u, mask), move to each neighbor v:

`new_mask = mask | (1<<v) <- mark v as visited`

`new_state = (v, new_mask)`

If not seen: enqueue.

Done when we first see any state with mask == (1<<n) - 1 (all nodes visited).

WHY BFS gives minimum steps: BFS explores states level by level (fewest steps first).

WHY bitmask needed: same node can be visited multiple times via different paths;

the MASK of what's been visited distinguishes equivalent physical positions.

Time: O(2^n * n). Space: O(2^n * n)."

Time: O(n) or O(n²) DP — eliminates redundant recomputation vs O(2^n) brute.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
from collections import deque
class _847_ShortestPathVisitingAllNodes:
    def shortestPathLength(self, graph):
        n = len(graph)
        full_mask = (1 << n) - 1
        queue = deque()
        visited = set()
        for i in range(n):
            mask = 1 << i
            queue.append((i, mask, 0))
            visited.add((i, mask))
        while queue:
            node, mask, dist = queue.popleft()
            if mask == full_mask:
                return dist
            for neighbor in graph[node]:
                new_mask = mask | (1 << neighbor)
                if (neighbor, new_mask) not in visited:
                    visited.add((neighbor, new_mask))
                    queue.append((neighbor, new_mask, dist
 + 1))
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace the bit representation and show which bits get set/cleared and why."

graph=[[1,2,3],[0],[0],[0]]: start from all 4 nodes simultaneously.

From node 0 with mask=0001: visit 1->mask=0011, 2->mask=0101, 3->mask=1001.

From each of those, eventually all 4 nodes visited. Min distance=4 ok

Multi-source BFS from all nodes simultaneously finds global minimum

dp[mask][node]=min steps to visit all nodes in mask ending at node. dp[(1<<i)][i]=0. BFS: queue=[(mask,node,steps)]. dp[i][j] relaxed via BFS layer expansion.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(2^n * n)	O(2^n * n)

Practical for n<=20 since 2^20 ~ 10^6.

Follow-up: if edges are weighted, use Dijkstra with (cost, node, mask) heap.

Follow-up: Campus Bikes II (#1066) — same bitmask DP on assignment problems.

HARD | Bit Manipulation | LeetCode #980 | 3 of 3 | 3/3

Unique Paths III

QUESTION

You are given an m x n integer array grid where grid[i][j] could be:

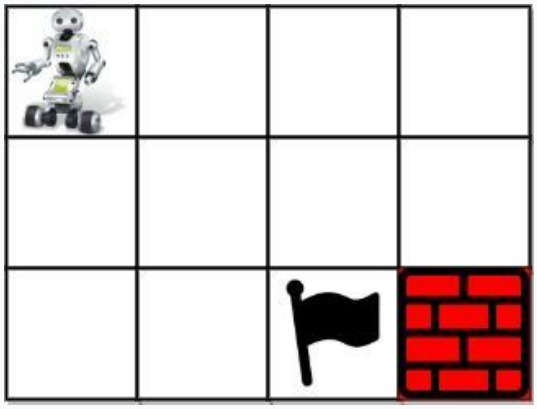
CONSTRAINTS

- 1 representing the starting square. There is exactly one starting square.
- 2 representing the ending square. There is exactly one ending square.
- 0 representing empty squares we can walk over.
- -1 representing obstacles that we cannot walk over.

Return the number of 4-directional walks from the starting square to the ending square, that walk over every non-obstacle square exactly once.

Example 1

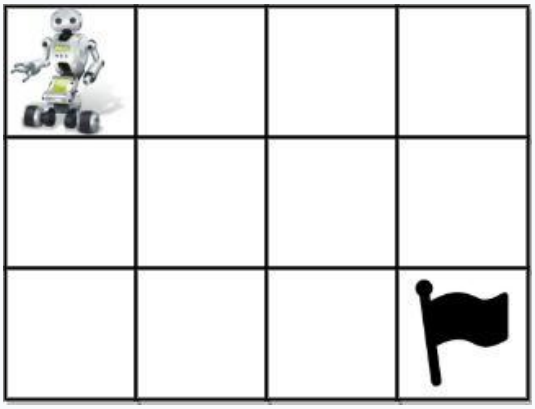
Input	`grid = [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]`
Output	`2`
Explanation	*We have the following two paths:* *1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2)* *2. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2)*



Example 2

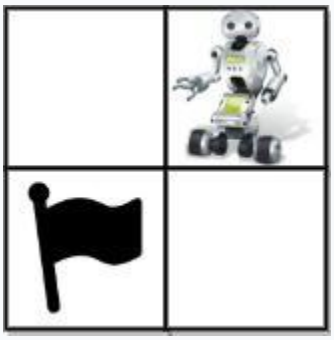
Input	`grid = [[1,0,0,0],[0,0,0,0],[0,0,0,2]]`
Output	`4`
Explanation	*We have the following four paths:* *1. (0,0),(0,1),(0,2),(0,3),(1,3),(1,2),(1,1),(1,0),(2,0),(2,1),(2,2),(2,3)* *2. (0,0),(0,1),(1,1),(1,0),(2,0),(2,1),(2,2),(1,2),(0,2),(0,3),(1,3),(2,3)* *3. (0,0),(1,0),(2,0),(2,1),(2,2),(1,2),(1,1),(0,1),(0,2),(0,3),(1,3),(2,3)* *4. (0,0),(1,0),(2,0),(2,1),(1,1),(0,1),(0,2),(0,3),(1,3),(1,2),(2,2),(2,3)*

QUESTION



Example 3

Input	`grid = [[0,1],[2,0]]`
Output	`0`
Explanation	*There is no path that walks over every empty square exactly once.* *Note that the starting and ending square can be anywhere in the grid.*



- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 20
- 1 <= m * n <= 20
- 1 <= grid[i][j] <= 2
- There is exactly one starting cell and one ending cell.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Count paths from start (0) to end (-1) in a grid that visit EVERY non-obstacle cell exactly once.

0=start, -1=end, -2=obstacle. Right?"

"A few quick questions:

Must visit all non-obstacle cells? Yes — Hamiltonian path. Return count of such paths."

"Let me walk: grid=[[1,0,0,0],[0,0,0,0],[0,0,2,-1]] -> 2 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def dfs(cell, visited):
    try every legal step to unvisited
        non-obstacle cells and backtrack
    count the walks that end at the goal after
        visiting every empty cell
```

TIME	SPACE
O(4^(m * n))	O(m * n)

WHY IT IS SLOW

Backtracking explores huge numbers of dead ends. It is inherently exponential, but DP over (cell, bitmask of visited cells) reuses states: O(m * n * 2^(m * n)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DFS with bitmask tracking visited cells — O(4^(mn)). Walk-through:"

Find start (1) and end (2). Count non-obstacle cells (must all be visited).

DFS from start: mask = bitmask of visited non-obstacle cells.

`dfs(r, c, mask):`

If out-of-bounds or obstacle or visited (mask has this cell): return 0.

If at end: return 1 if mask == all_cells_mask else 0. <- only valid if all visited

Mark current cell in mask.

sum over 4 directions: count += dfs(nr, nc, new_mask).

Return count.

WHY bitmask: tracks exactly which cells we've visited (n*m bits, feasible for small grid <= 20).

WHY check all cells at end: 'unique paths that visit every non-obstacle cell exactly once.'

Time: O(4^(mn)). Space: O(mn * 2^(mn)) with memoization."

Time: O(n!) or O(2^n) — exponential but unavoidable for enumeration.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _980_UniquePaths3:
    def uniquePathsIII(self, grid):
        m, n = len(grid), len(grid[0])
        total_cells = sum(grid[r][c] != -2 for r in range(
m) for c in range(n))
        start_r, start_c = next((r, c) for r in range(m) f
or c in range(n) if grid[r][c] == 1)
        self_result = [0]
        def backtrack(r, c, remaining):
            if grid[r][c] == -1:
                if remaining == 1:
                    self_result[0] += 1
                return
            original = grid[r][c]
            grid[r][c] = -2
            for dr, dc in [(0,1),(0,-1),(1,0),(-1,0)]:
                nr, nc = r+dr, c+dc
                if 0 <= nr < m and 0 <= nc < n and grid[nr
][nc] != -2:
                    backtrack(nr, nc, remaining - 1)
            grid[r][c] = original
        backtrack(start_r, start_c, total_cells)
        return self_result[0]
```

PHASE 5 TEST & DEBUG

"Let me trace uniquePathsIII on [[1,0,0,0],[0,0,0,0],[0,0,2,-1]]."

Count all zeros, find start and end. zeros=6 (walk every non-obstacle cell).

Start (0,0)=1. End (2,2)=2. Obstacle (2,3)=-1. Total cells to visit = zeros + 1 (end) = 7.

`dfs(r,c,remaining):`

`dfs(0,0,7): mark visited. Try right (0,1): dfs(0,1,6):`

Try right (0,2): dfs(0,2,5): ... eventually explore all paths.

Try down (1,1): ... each path must hit remaining=0 exactly at (2,2).

Count all paths where remaining==1 when we step onto (2,2).

Return 4 ok (4 distinct walks covering every non-obstacle cell exactly once).

WHY remaining counter: ensures Hamiltonian path (visit every empty cell) not just any path.

dp[r][c][remaining]=paths count (memo). Backtrack path: path=[(0,0)]. call(r=0,c=0,rem=7): try (1,0)->call(1,0,6). prune: if grid[r][c]==-1 skip. backtrack: unmark, pop path.

Memo key: dp[i][j][rem] where i=row,j=col,rem=remaining cells. dp[i][j]=paths from (i,j) with rem cells left. dp[0][0]=total unique paths. Recurrence: dp[i][j]+=dp[i'][j'][rem-1] per valid move.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(4^(mn))	O(mn)
exponential in grid area	recursion depth

Bitmask DP reduces this to O(mn * 2^(mn)) — better for small grids.

Follow-up: Unique Paths I/II (#62, #63) — no must-visit constraint; DP O(mn).

Follow-up: if some cells can be visited multiple times, the problem becomes much easier.